DATA STRUCTURES & ALGORITHMS

MODULE II - DATA STRUCTURE TYPES

Arrays, Stacks, Infix/Postfix/Prefix Expressions, Queues, and Circular Queues
with implementation, dry runs, operation analysis, Python programs, and exam-ready notes.

Syllabus covered

- 2.1 Arrays - implementation and analysis of basic array operations
- 2.1 Stack - implementation and analysis of push and pop operations
- 2.2 Infix, postfix, and prefix expressions
- 2.3 Queue - implementation and analysis of basic queue operations
- 2.4 Circular Queue - implementation and analysis

Python 3 | Detailed learning + examination notes

Module II Learning Objectives

- Understand the memory and indexing model of arrays and analyze their basic operations.
- Understand the LIFO principle of stacks and implement push, pop, peek, and related checks.
- Recognize infix, postfix, and prefix notation and perform expression conversion and evaluation using stacks.
- Understand FIFO queues and implement enqueue, dequeue, front/peek, and traversal.
- Understand why a simple array queue can waste space and how a circular queue reuses it.
- State the time and space complexity of the major operations covered in this module.

Notation note: n denotes the current number of elements. For fixed-capacity structures, capacity denotes the maximum number of elements that can be stored.

2.1 Arrays

2.1.1 Definition

An array is a linear data structure that stores elements in indexed positions. In a traditional low-level array, elements of the same type occupy contiguous memory locations. This makes direct index access efficient. Python's built-in list is a dynamic array-like structure that stores references and automatically resizes, so it is convenient for learning array operations even though its internal representation differs from a fixed C-style array.

2.1.2 Array Representation

Index	**0**	**1**	**2**	**3**	**4**
Value	10	20	30	40	50

For a zero-based array A, the first element is A[0] and the last element is A[n-1]. If each fixed-size element occupies w bytes and the base address is B, a conceptual address formula is Address(A[i]) = B + i*w.

2.1.3 Basic Array Operations

Operation	**What it does**	**Typical time**
Access by index	Read A[i]	O(1)
Update by index	Replace A[i]	O(1)
Traversal	Visit all n elements	O(n)
Linear search	Check elements until target is found	O(n) worst case
Insert at end	Append after last element	O(1) amortized for Python list
Insert at beginning/middle	Shift later elements right	O(n)
Delete from end	Remove last element	O(1) for Python list
Delete from beginning/middle	Shift later elements left	O(n)

2.1.4 Access and Update

```
numbers = [10, 20, 30, 40, 50]


print(numbers[2])
numbers[2] = 35
print(numbers)
```

Output

```
30
[10, 20, 35, 40, 50]
```

Direct indexing does not require scanning earlier elements, so access and update are O(1).

2.1.5 Traversal

```
numbers = [10, 20, 30, 40, 50]

for value in numbers:
    print(value, end=" ")
```

Output

```
10 20 30 40 50
```

Every element is visited once, so traversal is O(n).

2.1.6 Linear Search

```
def linear_search(a, target):
    for i in range(len(a)):
        if a[i] == target:
            return i
    return -1

a = [12, 7, 25, 9, 18]
print(linear_search(a, 9))
print(linear_search(a, 100))
```

Output

```
3
-1
```

Case	Comparisons	Complexity
Best	Target at first position	O(1)
Worst	Target last or absent	O(n)
Average	Position depends on input distribution	O(n)

2.1.7 Insertion in an Array

To insert at index k in a fixed array with free capacity, elements from k through n-1 are shifted one position to the right, then the new value is written at k.

```
a = [10, 20, 30, 40]
position = 2
value = 25
```

```
a.insert(position, value)
print(a)
```

Output

```
[10, 20, 25, 30, 40]
```

Insertion near the beginning may shift almost n elements, so arbitrary-position insertion is O(n). Appending to a Python list is amortized O(1), although an occasional resize can cost O(n).

2.1.8 Deletion from an Array

```
a = [10, 20, 25, 30, 40]
deleted = a.pop(2)

print("Deleted:", deleted)
print(a)
```

Output

```
Deleted: 25
[10, 20, 30, 40]
```

Deleting from an arbitrary index requires later elements to shift left, so it is O(n). Removing the last Python-list element with pop() is O(1).

2.1.9 Manual Fixed-Capacity Array Insertion

```
capacity = 6
a = [10, 20, 30, 40, None, None]
n = 4
position = 2
value = 25

for i in range(n, position, -1):
    a[i] = a[i - 1]

a[position] = value
n += 1

print(a[:n])
```

Output

```
[10, 20, 25, 30, 40]
```

Dry run: 40 moves from index 3 to 4; 30 moves from index 2 to 3; then 25 is placed at index 2.

2.1.10 Advantages and Limitations of Arrays

Advantages	Limitations
Fast direct indexing	Insertion/deletion in the middle can require shifting

Simple traversal	Traditional fixed arrays have fixed capacity
Good locality in true contiguous arrays	Need an index or search strategy to find an unknown value
Useful foundation for stacks and queues	Python list may resize and stores references rather than raw homogeneous values

2.1 Stack

2.1.11 Definition and LIFO Principle

A stack is a linear data structure that follows LIFO: Last In, First Out. Insertion and deletion occur at one end called TOP. The insertion operation is PUSH and the deletion operation is POP.

Bottom			**Top**
10	20	30	40

If 50 is pushed, it becomes the new top. A pop then removes 50 first.

2.1.12 Stack Operations

Operation	**Meaning**	**Array-based time**
push(x)	Insert x at top	O(1) normally; amortized O(1) for Python list
pop()	Remove and return top	O(1)
peek()/top()	Read top without removing it	O(1)
is_empty()	Check whether stack has no elements	O(1)
size()	Return number of stored elements	O(1) when size is maintained/known
traversal	Visit all elements	O(n)

2.1.13 Stack Implementation Using Python List

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, value):
        self.items.append(value)

    def pop(self):
        if self.is_empty():
            return "Stack Underflow"
        return self.items.pop()

    def peek(self):
        if self.is_empty():
            return "Stack is empty"
        return self.items[-1]
```

```
    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

stack = Stack()
stack.push(10)
stack.push(20)
stack.push(30)

print("Stack:", stack.items)
print("Top:", stack.peek())
print("Popped:", stack.pop())
print("Stack:", stack.items)
```

Output

```
Stack: [10, 20, 30]
Top: 30
Popped: 30
Stack: [10, 20]
```

2.1.14 PUSH Operation - Algorithm and Analysis

For a fixed-size array stack, TOP commonly starts at -1.

```
PUSH(STACK, MAX, TOP, ITEM)
1. If TOP == MAX - 1:
       report "Stack Overflow"
       stop
2. TOP = TOP + 1
3. STACK[TOP] = ITEM
4. return TOP
```

PUSH performs a constant number of operations independent of n, so a fixed-array push is O(1). In a dynamic Python list, append is amortized O(1).

2.1.15 POP Operation - Algorithm and Analysis

```
POP(STACK, TOP)
1. If TOP == -1:
       report "Stack Underflow"
       stop
2. ITEM = STACK[TOP]
3. TOP = TOP - 1
4. return ITEM, TOP
```

POP also performs constant work, so its time complexity is O(1).

2.1.16 Overflow and Underflow

- Stack Overflow: attempting to push into a full fixed-capacity stack.
- Stack Underflow: attempting to pop from an empty stack.
- Dynamic structures such as Python lists do not have a fixed logical capacity chosen by the programmer, but memory is still finite.

2.1.17 Applications of Stack

- Function calls and recursion call stack.
- Undo/redo mechanisms.
- Balanced-parentheses checking.
- Expression conversion: infix to postfix/prefix.
- Postfix and prefix evaluation.
- Depth-first search and backtracking.

2.2 Infix, Postfix, and Prefix

2.2.1 Definitions

Notation	Operator position	Example for A + B	Also called
Infix	Between operands	A + B	Conventional notation
Postfix	After operands	A B +	Reverse Polish notation
Prefix	Before operands	+ A B	Polish notation

Postfix and prefix remove the need for parentheses when the expression is correctly formed because operator placement determines evaluation order.

2.2.2 Operator Precedence and Associativity

Operator	Meaning	Precedence	Associativity
^	Exponent	Highest	Right to left
* / %	Multiply, divide, modulo	Next	Left to right
+ -	Add, subtract	Lower	Left to right

Parentheses override normal precedence in infix expressions.

2.2.3 Simple Conversions

Infix	Postfix	Prefix
A + B	A B +	+ A B
A + B * C	A B C * +	+ A * B C
(A + B) * C	A B + C *	* + A B C
A * (B + C)	A B C + *	* A + B C
(A + B) * (C - D)	A B + C D - *	* + A B - C D

2.2.4 Infix to Postfix Using a Stack

1. Scan the infix expression from left to right.
2. If the token is an operand, append it to the output.
3. If it is '(', push it.
4. If it is ')', pop operators to output until '(' is found; discard the parentheses.
5. If it is an operator, pop higher-precedence operators from the stack. For equal precedence, associativity determines whether to pop.
6. Push the current operator.
7. After scanning all tokens, pop remaining operators to output.

Dry Run: A + B * C

Token	Action	Stack	Postfix output
A	Output operand	empty	A
+	Push +	+	A
B	Output operand	+	A B
*	Higher precedence than +; push	+ *	A B
C	Output operand	+ *	A B C
End	Pop all	empty	A B C * +

Therefore, A + B * C becomes A B C * +.

2.2.5 Python: Infix to Postfix

```
def infix_to_postfix(expression):
    precedence = {'+': 1, '-': 1, '*': 2, '/': 2, '%': 2, '^': 3}
    stack = []
    output = []

    for token in expression.split():
        if token.isalnum():
            output.append(token)
        elif token == '(':
            stack.append(token)
        elif token == ')':
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            stack.pop()
        else:
            while (stack and stack[-1] != '(' and
                   (precedence[stack[-1]] > precedence[token] or
                    (precedence[stack[-1]] == precedence[token] and token != '^'))):
                output.append(stack.pop())
            stack.append(token)

    while stack:
        output.append(stack.pop())
```

```
    return ' '.join(output)

print(infix_to_postfix("A + B * C"))
print(infix_to_postfix("( A + B ) * C"))
```

Output

```
A B C * +
A B + C *
```

This educational version expects tokens separated by spaces. The algorithm processes each token and uses a stack for operators; with standard stack operations, the conversion is O(n) time and O(n) auxiliary space.

2.2.6 Postfix Evaluation

8. Scan postfix tokens from left to right.
9. If a token is an operand, push it.
10. If it is an operator, pop the right operand, then pop the left operand.
11. Apply the operator and push the result.
12. After the final token, the stack's single value is the answer.

Dry Run: 2 3 4 * +

Token	Action	Stack
2	Push 2	[2]
3	Push 3	[2, 3]
4	Push 4	[2, 3, 4]
*	3 * 4 = 12	[2, 12]
+	2 + 12 = 14	[14]

Result = 14.

2.2.7 Python: Postfix Evaluation

```
def evaluate_postfix(expression):
    stack = []

    for token in expression.split():
        if token.lstrip('-').isdigit():
            stack.append(int(token))
        else:
            right = stack.pop()
            left = stack.pop()

            if token == '+':
                stack.append(left + right)
            elif token == '-':
                stack.append(left - right)
            elif token == '*':
                stack.append(left * right)
```

```
            elif token == '/':
                stack.append(left / right)

    return stack.pop()

print(evaluate_postfix("2 3 4 * +"))
```

Output

```
14
```

2.2.8 Prefix Evaluation

Prefix evaluation is similar, but scan from right to left. Push operands. When an operator is found, pop the first operand, then the second operand, apply the operator, and push the result.

Dry Run: + 2 * 3 4

Scanned token (right to left)	Action	Stack
4	Push	[4]
3	Push	[4, 3]
*	3 * 4 = 12	[12]
2	Push	[12, 2]
+	2 + 12 = 14	[14]

Result = 14.

2.2.9 Python: Prefix Evaluation

```
def evaluate_prefix(expression):
    stack = []

    for token in reversed(expression.split()):
        if token.lstrip('-').isdigit():
            stack.append(int(token))
        else:
            left = stack.pop()
            right = stack.pop()

            if token == '+':
                stack.append(left + right)
            elif token == '-':
                stack.append(left - right)
            elif token == '*':
                stack.append(left * right)
            elif token == '/':
                stack.append(left / right)

    return stack.pop()
```

```
print(evaluate_prefix("+ 2 * 3 4"))
```

Output

```
14
```

2.2.10 Expression Complexity

Task	Time	Auxiliary space
Infix to postfix	O(n)	O(n)
Postfix evaluation	O(n)	O(n) worst case
Prefix evaluation	O(n)	O(n) worst case

2.3 Queue

2.3.1 Definition and FIFO Principle

A queue is a linear data structure that follows FIFO: First In, First Out. Insertion occurs at the REAR and deletion occurs at the FRONT.

Front			Rear
10	20	30	40

If 50 is enqueued, it is added at the rear. A dequeue removes 10 first.

2.3.2 Basic Queue Operations

Operation	Meaning	Efficient array/deque implementation
enqueue(x)	Insert x at rear	O(1)
dequeue()	Remove and return front	O(1)
front()/peek()	Read front item	O(1)
rear()	Read last item	O(1)
is_empty()	Check whether queue is empty	O(1)
traversal	Visit all current elements	O(n)

Important Python note: list.pop(0) is O(n) because remaining references shift left. For an efficient general-purpose Python queue, collections.deque provides O(1) append and popleft operations.

2.3.3 Efficient Python Queue Using deque

```
from collections import deque

queue = deque()

queue.append(10)
queue.append(20)
queue.append(30)
```

```
print("Queue:", list(queue))
print("Front:", queue[0])
print("Dequeued:", queue.popleft())
print("Queue:", list(queue))
```

Output

```
Queue: [10, 20, 30]
Front: 10
Dequeued: 10
Queue: [20, 30]
```

2.3.4 Fixed Array Queue with FRONT and REAR

In a simple linear array queue of capacity MAX, FRONT and REAR track the active region. One common convention starts both at -1.

```
ENQUEUE(Q, MAX, FRONT, REAR, ITEM)
1. If REAR == MAX - 1:
       report "Queue Overflow"
       stop
2. If FRONT == -1:
       FRONT = 0
3. REAR = REAR + 1
4. Q[REAR] = ITEM

DEQUEUE(Q, FRONT, REAR)
1. If FRONT == -1 or FRONT > REAR:
       report "Queue Underflow"
       stop
2. ITEM = Q[FRONT]
3. FRONT = FRONT + 1
4. If FRONT > REAR:
       FRONT = REAR = -1
5. return ITEM
```

With indices rather than shifting the array, enqueue and dequeue are O(1). However, once REAR reaches the final physical index, unused positions before FRONT cannot be reused by this simple linear design. Circular queues solve that problem.

2.3.5 Queue Dry Run

Operation	Front	Rear	Logical queue
Initial	-1	-1	empty
enqueue(10)	0	0	10
enqueue(20)	0	1	10, 20
enqueue(30)	0	2	10, 20, 30
dequeue()	1	2	20, 30
enqueue(40)	1	3	20, 30, 40

2.3.6 Queue Applications

- CPU and process scheduling.
- Printer/job scheduling.
- Breadth-first search (BFS).
- Request and message buffering.
- Customer/service waiting systems.
- Producer-consumer systems.

2.3.7 Stack vs Queue

Feature	Stack	Queue
Rule	LIFO	FIFO
Insertion	Push at top	Enqueue at rear
Deletion	Pop from top	Dequeue from front
Main pointers	TOP	FRONT and REAR
Example	Undo history	Waiting line / BFS

2.4 Circular Queue

2.4.1 Why Circular Queue?

A circular queue treats the array as if the position after the last index is index 0. This allows empty positions created by dequeue operations to be reused. The wrap-around is implemented using modulo arithmetic.

2.4.2 Core Conditions

For an array of capacity N:

Condition/Update	Formula
Move rear forward	rear = (rear + 1) % N
Move front forward	front = (front + 1) % N
Full condition	(rear + 1) % N == front
Empty condition in this convention	front == -1

This convention deliberately leaves one slot unavailable to distinguish full from empty using only front and rear. Other valid implementations keep a separate count and can use all N slots.

2.4.3 Circular Queue Example

Capacity = 5. Suppose the queue contains items at indices 2, 3, and 4. After another enqueue, rear wraps from 4 to 0 because (4 + 1) % 5 = 0, provided index 0 is available.

Index	0	1	2	3	4
Before wrap	empty	empty	30 (front)	40	50 (rear)
After enqueue 60	60 (rear)	empty	30 (front)	40	50

2.4.4 Circular Queue Enqueue Algorithm

```
CIRCULAR_ENQUEUE(Q, N, FRONT, REAR, ITEM)
```

```
1. If (REAR + 1) % N == FRONT:
       report "Queue Overflow"
       stop
2. If FRONT == -1:
       FRONT = 0
       REAR = 0
3. Else:
       REAR = (REAR + 1) % N
4. Q[REAR] = ITEM
```

The operation performs a fixed amount of work, so enqueue is O(1).

2.4.5 Circular Queue Dequeue Algorithm

```
CIRCULAR_DEQUEUE(Q, N, FRONT, REAR)
1. If FRONT == -1:
       report "Queue Underflow"
       stop
2. ITEM = Q[FRONT]
3. If FRONT == REAR:
       FRONT = REAR = -1
4. Else:
       FRONT = (FRONT + 1) % N
5. return ITEM
```

Dequeue is also O(1).

2.4.6 Python Circular Queue Implementation

```
class CircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = [None] * capacity
        self.front = 0
        self.rear = 0
        self.count = 0

    def is_empty(self):
        return self.count == 0

    def is_full(self):
        return self.count == self.capacity

    def enqueue(self, value):
        if self.is_full():
            return "Queue Overflow"
```

```
        self.data[self.rear] = value
        self.rear = (self.rear + 1) % self.capacity
        self.count += 1
        return "Inserted"

    def dequeue(self):
        if self.is_empty():
            return "Queue Underflow"

        value = self.data[self.front]
        self.data[self.front] = None
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return value

    def display(self):
        result = []
        index = self.front

        for _ in range(self.count):
            result.append(self.data[index])
            index = (index + 1) % self.capacity

        return result

q = CircularQueue(4)

print(q.enqueue(10))
print(q.enqueue(20))
print(q.enqueue(30))
print(q.display())

print("Removed:", q.dequeue())
print(q.enqueue(40))
print(q.enqueue(50))
print(q.display())
```

Output

```
Inserted
Inserted
Inserted
[10, 20, 30]
Removed: 10
```

```
Inserted
Inserted
[20, 30, 40, 50]
```

This implementation uses a count variable, so all capacity slots can be used. enqueue and dequeue are O(1); display is O(n).

2.4.7 Circular Queue Dry Run

Operation	Front	Rear (next insertion)	Count	Logical contents
Start	0	0	0	[]
enqueue 10	0	1	1	[10]
enqueue 20	0	2	2	[10, 20]
enqueue 30	0	3	3	[10, 20, 30]
dequeue	1	3	2	[20, 30]
enqueue 40	1	0	3	[20, 30, 40]
enqueue 50	1	1	4	[20, 30, 40, 50]

2.4.8 Linear Queue vs Circular Queue

Feature	Linear array queue	Circular queue
Arrangement	Conceptually straight	Last index connects back to first
Reuse freed front positions	Not in the simplest non-shifting implementation	Yes
Next-index update	index + 1	(index + 1) % capacity
Enqueue/dequeue	O(1) with index tracking	O(1)
Main benefit	Simple	Better fixed-array space utilization

2.4.9 Applications of Circular Queue

- Round-robin CPU scheduling.
- Fixed-size buffers and streaming buffers.
- Keyboard/network/device buffers.
- Repeated cyclic resource allocation.
- Producer-consumer systems with bounded buffers.

Module II Complexity Summary

Data structure / task	Operation	Time complexity
Array	Access/update by index	O(1)
Array	Traversal	O(n)
Array	Linear search	O(n) worst case
Array	Insert/delete arbitrary position	O(n)
Python list	Append	O(1) amortized
Stack	Push	O(1); Python list amortized O(1)
Stack	Pop/peek	O(1)
Expression	Infix to postfix	O(n)

Expression	Postfix/prefix evaluation	O(n)
Queue	Enqueue/dequeue with suitable implementation	O(1)
Circular Queue	Enqueue/dequeue	O(1)
Circular Queue	Traversal/display	O(n)

Important Exam Differences

Pair	Key difference
Array vs Stack	Array allows indexed access; stack restricts insertion/deletion to TOP.
Stack vs Queue	Stack is LIFO; queue is FIFO.
Infix vs Postfix	Infix places operator between operands; postfix places it after operands.
Postfix vs Prefix	Postfix is usually scanned left-to-right for evaluation; prefix right-to-left.
Linear Queue vs Circular Queue	Circular queue wraps indices and reuses freed positions.
Overflow vs Underflow	Overflow means insertion cannot be performed due to full capacity; underflow means deletion is attempted on an empty structure.

Exam-Ready Short Definitions

Array: A linear indexed data structure used to store a collection of elements; traditional arrays store same-type elements in contiguous memory locations.

Stack: A linear data structure following LIFO, where insertion and deletion take place at the TOP.

Push: The stack operation used to insert an element at TOP.

Pop: The stack operation used to remove the TOP element.

Infix: Expression notation in which an operator appears between its operands, e.g., A + B.

Postfix: Expression notation in which an operator appears after its operands, e.g., A B +.

Prefix: Expression notation in which an operator appears before its operands, e.g., + A B.

Queue: A linear data structure following FIFO, where insertion occurs at REAR and deletion at FRONT.

Enqueue: The operation of inserting an element at the rear of a queue.

Dequeue: The operation of removing an element from the front of a queue.

Circular Queue: A queue in which the last array position is logically connected to the first so freed positions can be reused.

Practice Questions

13. Define an array. Explain access, traversal, search, insertion, and deletion with time complexities.

14. Why is direct array access O(1)?
15. Write an algorithm to insert an element at a given array position and analyze it.
16. Define a stack. Explain LIFO with an example.
17. Write and analyze PUSH and POP algorithms for an array stack.
18. Explain stack overflow and stack underflow.
19. Convert A + B * C to postfix and prefix.
20. Convert (A + B) * (C - D) to postfix and prefix.
21. Evaluate the postfix expression: 5 2 3 * +.
22. Evaluate the prefix expression: + 5 * 2 3.
23. Define a queue and explain FIFO.
24. Write algorithms for ENQUEUE and DEQUEUE and state their complexities.
25. Why can a simple linear array queue waste space?
26. Define a circular queue and explain modulo-based wrap-around.
27. Write circular queue enqueue and dequeue algorithms.
28. Compare stack, linear queue, and circular queue.
29. State practical applications of stacks, queues, and circular queues.

Selected Answers for Self-Check

Question	Answer
7	Postfix: A B C * + ; Prefix: + A * B C
8	Postfix: A B + C D - * ; Prefix: * + A B - C D
9	11
10	11
13	After FRONT moves forward, freed low-index positions cannot be reused if REAR only moves toward the last index.
14	Use modulo: next = (current + 1) % capacity.

Final Revision Checklist

- I can explain and analyze array access, traversal, search, insertion, and deletion.
- I can implement a stack and explain push, pop, overflow, and underflow.
- I can identify infix, postfix, and prefix expressions.
- I can convert infix to postfix using an operator stack.
- I can evaluate postfix and prefix expressions using a stack.
- I can implement a queue and explain front and rear.
- I understand why list.pop(0) is not an efficient Python queue operation.
- I can explain circular queue wrap-around using modulo.
- I can state full and empty conditions for the circular-queue convention being used.
- I can compare the time complexity of all major Module II operations.